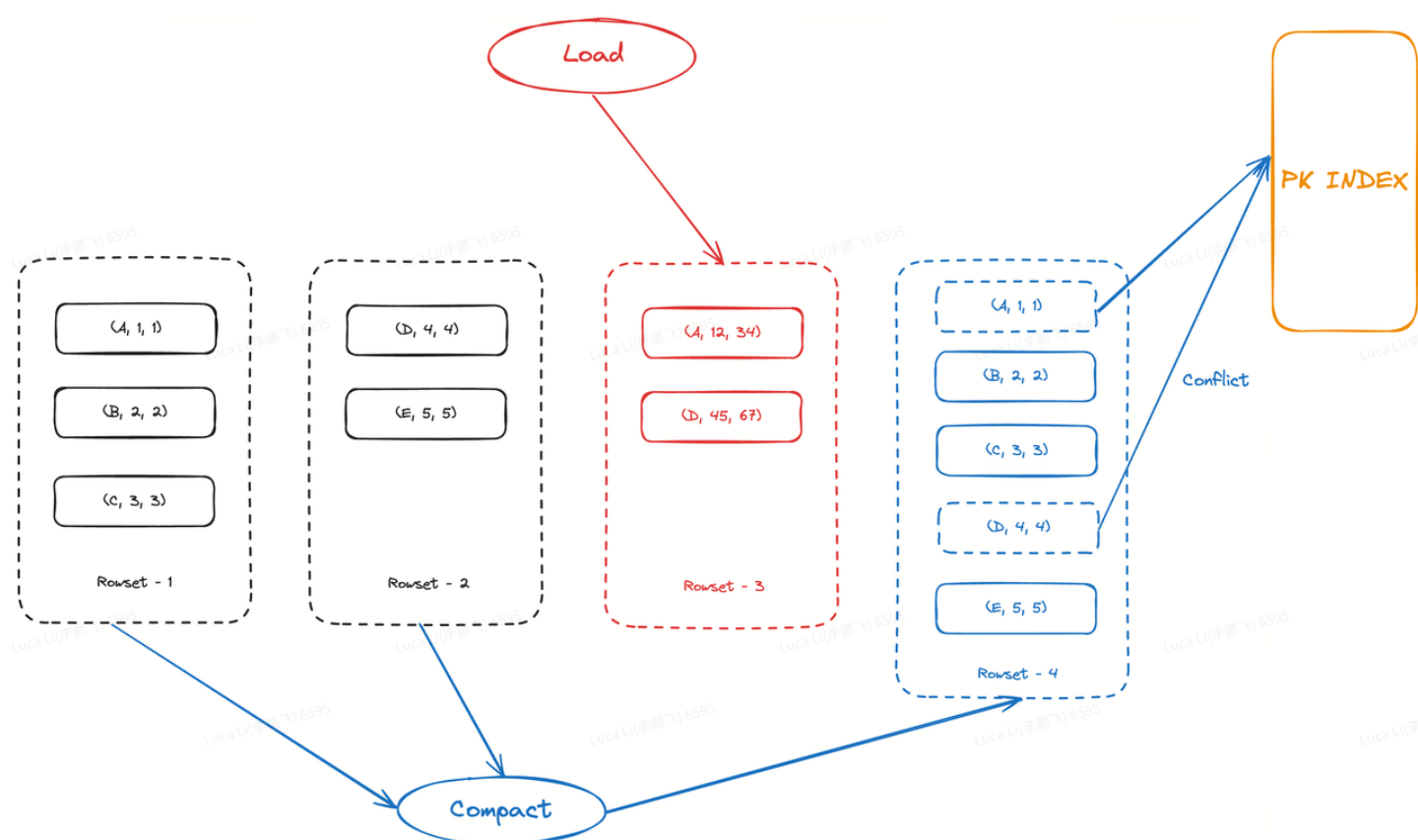


[RFC] PK表compaction事务publish & apply 执行优化方案

背景

PK表的compaction执行和导入是可以并发的，因此在publish & apply阶段需要解决compaction和导入并发冲突的问题。举个例子：



在上图中，rowset-1和rowset-2执行compact的过程中，插入了导入生成的rowset-3，他修改了行A和D，那最终compact生成的rowset-4中的，行A和D就需要被标记删除。

在目前已有的实现中，我们是通过pk index的try_replace接口来实现，检查output rowset中每一行是否该被标记删除，将未被删除的行更新到pk index中。

这里包含几个步骤：

1. 按segment粒度去加载PK列到内存。
2. 使用PK列去查询pk index，判断当前行是否是最新的行。
3. 如果是最新的行，则需要更新到pk index里。
4. 如果不是最新，说明有并发的导入事务，此时需要生成新的delete vector对该行进行标记删除。

存在的问题

目前的实现存在几个问题：

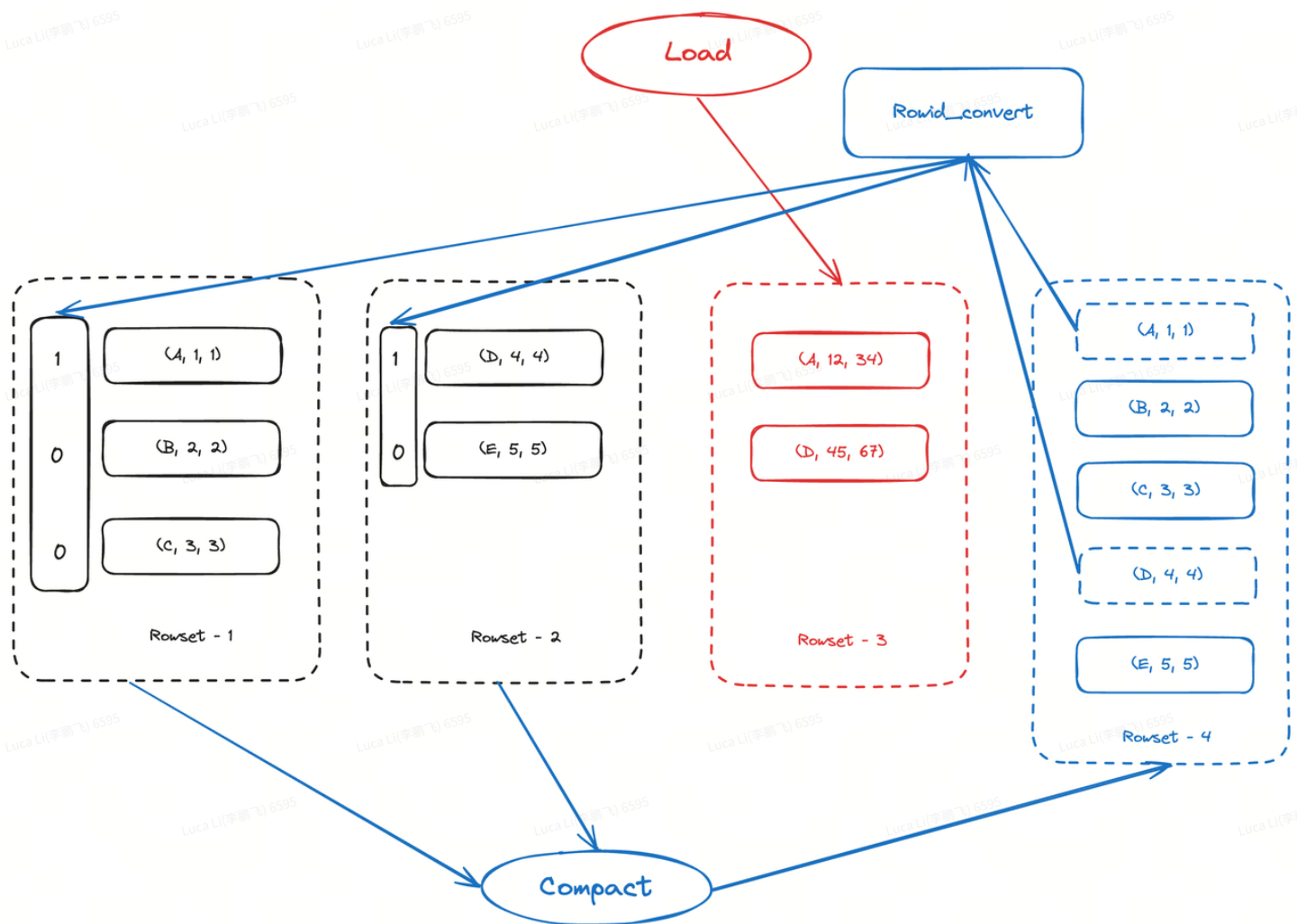
1. 因为compaction通常会带来大量行的合并，甚至是全量合并。因此会带来大量对pk index的读IO。
2. 内存占用不可控。因为我们是按segment粒度去加载PK列到内存，如果segment的压缩比很高的情况下，1G的segment的PK列加载出来，可能会放大达到5倍(**Abjayon这个客户真实案例**)!!!
 - a. 假设一个场景，用户是16C64G的机器，每个apply线程需要加载5G PK列(5倍放大)，则最高需要 $16 * 5GB = 80 GB$ 的内存，远超机器限制。
 - b. 那是否可以流式加载PK列？这么做可以节约内存，但是会导致多次查询pk index，导致进一步的读IO放大。问题1和问题2在现有方案下不可兼得。

优化方案介绍

新的方案中，compaction处理包含几个步骤：

1. 在compact执行过程中，生成一个包含output segment中每一行的rowid到其input segment中原始行rowid的映射关系的文件。
2. 在后续的publish & apply阶段中，流式加载output segment的PK列到内存，同时读取对应的input segment中原始行rowid。
3. 通过input segment中原始行rowid 去查询其目前最新的delete vector，来判断新output segment中每一行是否应该被标记删除。

如下图所示：



代码块

- 1 1. 在上图中, compact合并Rowset-1和Rowset-2的过程中, 生成的行映射关系文件的内容为:
- 2 1-0,
- 3 1-1,
- 4 1-2,
- 5 2-0,
- 6 2-1.
- 7
- 8 2. 在publish & apply的时候, 通过行映射关系文件中的原始行, 去反查delvec, 得到:
- 9 1-0 -> 被删除
- 10 1-1 -> 存在
- 11 1-2 -> 存在
- 12 2-0 -> 被删除
- 13 2-1 -> 存在
- 14
- 15 3. 因此最后, 为Rowset-4生成的delvec为 <1, 0, 0, 1, 0>

本方案可以, 在避免读取PK index的同时, 实现了流式PK列加载, 解决了现有方案存在的问题。

测试结果

读IO的优化效果，之前的文档中已经量化研究过，不再重复测试。 [PersistentIndex IO放大优化方案](#)

本测试主要聚焦，内存使用和compaction publish执行延迟的优化效果。为了更准确地对比优化对于compaction publish上执行延迟的提升，我们主要统计compaction publish在生成delvec并修改index这部分的耗时。

执行延迟改进效果

	Compaction publish耗时(ms) (生成delvec + 修改index)	
	优化前	优化后
1G + insert + 1/100	15416	7042
1G + upsert + 1/100	38144	16292
1G + upsert + 1/10000	8330	4557
2.5G + insert + 1/100	77074	29431
2.5G + upsert + 1/100	359913	71226
2.5G + upsert + 1/10000	142852	15576

可以看到，优化后的生成delvec并修改index的这部分耗时相比优化前，是有很大的提升的。继续再细化研究一下提升来着哪个阶段，以 `2.5G + insert + 1/100` 为例，更细粒度地研究优化前后各个阶段的耗时：

代码块

1

优化前：

2

总耗时（77074）= 读取segment（做了预加载）（0）+ 读取和修改index（77026）

3

4

优化后：

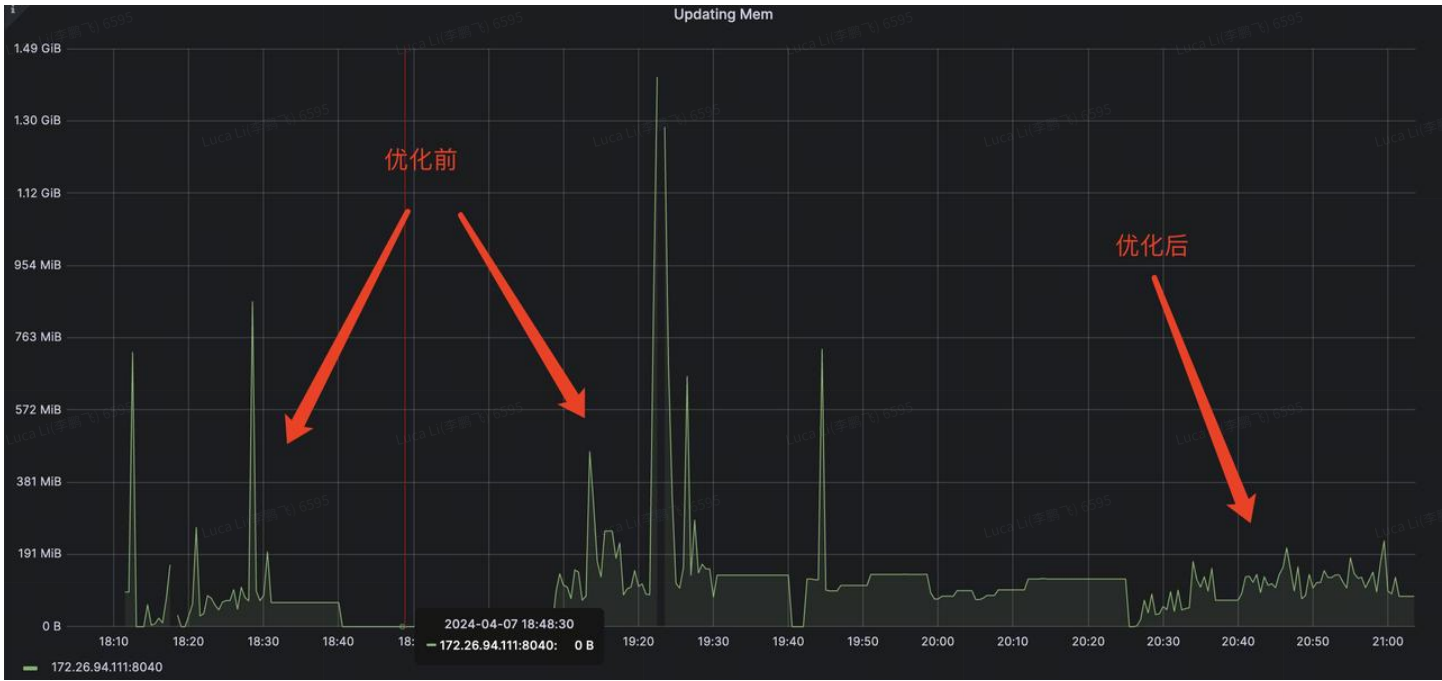
5

总耗时（29431）= 读取segment（4514）+ 修改index（20206）+ 读取delvec（2478）

可以看到，执行延迟的改进主要来自于减少了pk index的读取。

内存使用改进效果

通过监控内存的使用，可以看到，新的compaction publish实现内存占用更加平稳。



Luca Li(李鹏飞) 6595

Luca Li(李鹏飞) 6595

Luca Li(李鹏飞) 6595

Luca Li(李鹏飞) 6595

Luca Li(李鹏飞) 6595

Luca Li(李鹏飞) 6595

Luca Li(李鹏飞) 6595

Luca Li(李鹏飞) 6595

Luca Li(李鹏飞) 6595

Luca Li(李鹏飞) 6595

Luca Li(李鹏飞) 6595

Luca Li(李鹏飞) 6595

Luca Li(李鹏飞) 6595

Luca Li(李鹏飞) 6595

Luca Li(李鹏飞) 6595

Luca Li(李鹏飞) 6595